

UBCC 跨节点缓存一致性协议体系结构

目录

1. 方案概述
2. 总体体系结构
3. 核心组件设计
4. 全局一致性语义
5. 关键协议路径
6. 并发仲裁与活性机制
7. Outer 协议性能分析
 - 7.1 比较范围与评价维度
 - 7.2 状态集合的性能影响
 - 7.3 Outer 组织方式
 - 7.4 远程读与所有权迁移
 - 7.5 Shared-to-Writer 与失效扇出
 - 7.6 吞吐、流量与排队
 - 7.7 目录与瞬态存储开销
 - 7.8 状态集合与组织方式的组合选择
 - 7.9 综合结论
8. 集成边界
9. 总结
- 附录 A 消息与状态速查表
- 附录 B EP-RNF 仲裁规则
- 附录 C 术语表

1. 方案概述

1.1 方案定位

UBCC 方案是面向多节点系统的跨节点缓存一致性体系结构。该方案在节点内 gem5 CHI 一致性域之上，构建独立的跨节点目录与仲裁层，并通过 EP 接入现有处理器缓存层级。

UBCC 方案的核心价值是将全局目录、跨节点权限仲裁和元数据容量管理从节点内 HN-F 资源域中分离出来，使节点内一致性与跨节点一致性保持清晰边界：

- 1. 节点内缓存层级继续使用现有 gem5 CHI 机制；
- 2. 跨节点 sharer、owner 和权限迁移由 Home UBCC 控制器统一管理；
- 3. ResidentDir 与 H64 Backstore 组成分层元数据体系；
- 4. EP-RNF、EP-SNF 与 UBAdapter 负责两个一致性域之间的协议衔接。

1.2 设计目标

UBCC 方案围绕以下目标进行设计：

- 协议解耦：全局目录与节点内 CHI 状态机职责分离；
- 容量扩展：在固定 SRAM 预算下提升等效追踪容量；
- 精确仲裁：依据全局目录状态选择 owner、sharer 和失效目标；
- 事务收敛：通过 epoch、reqId、两阶段提交和幂等处理保证同址事务有序完成；
- 多拓扑适配：支持多节点、多 Socket 和跨节点路由组织；
- 工程集成：以模块化接口接入 ubsim 环境。

1.3 交付结论

当前交付实现已形成完整的跨节点一致性数据通路和控制通路，覆盖远程读、所有权迁移、共享转写者、

写回、逐出以及目录换入换出等关键路径。方案通过分层验证确认协议状态安全、消息幂等和事务收敛，并在正式性能验收中达到三项正式指标。

1.4 结论与范围

项目	结论	适用范围
架构	独立 Outer 域、分层目录和 EP 接入路径已形成	当前交付实现及已验证拓扑
正确性	关键安全、活性与可恢复传输故障路径通过分层验证	形式化模型、定向验证与端到端执行所覆盖的机制
性能	三项正式验收指标达到门槛	既定 workload、统一完成边界与参考模型条件
范围外	不作本次交付能力主张	永久 Home 故障在线恢复和端口级 Switch 微体系结构

2. 总体体系结构

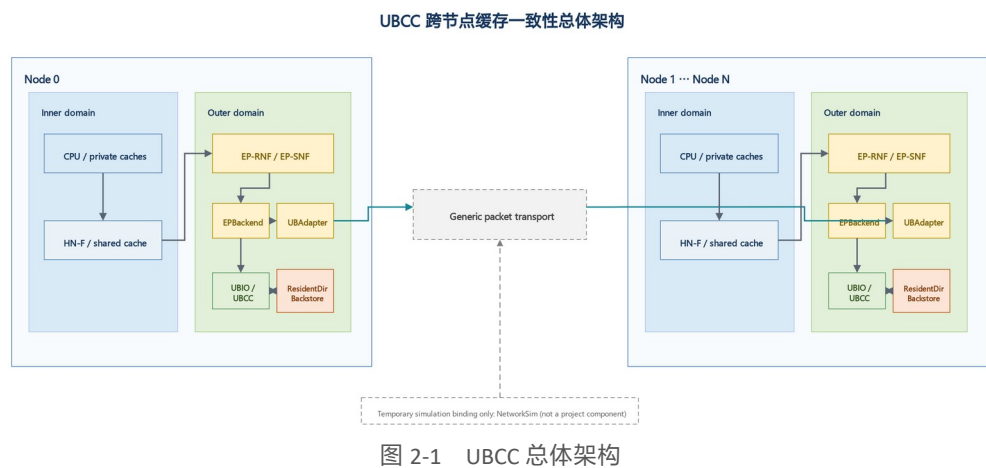
2.1 Inner 域与 Outer 域

UBCC 方案将系统划分为两个协同的一致性域：

- Inner 域：节点内 gem5 CHI 一致性域，包括 CPU Cache、HN-F 及节点内 snoop 路径；
- Outer 域：由 Home UBCC 控制器协同实现的跨节点一致性层，负责全局目录、权限仲裁、Recall 和 Invalidate 收敛。

节点内请求经 EP 转换为 Outer 请求并发送给目标地址的 Home UBCC 控制器；Home UBCC 控制器根据全局目录状态完成权限判断，并经跨节点通信平面交换一致性消息。

本文使用“UBCC 方案”表示完整体系结构，“UBCC 控制器”表示全局目录与仲裁组件，“Home UBCC 控制器”表示目标地址映射到的具体控制器实例，“Outer 协议”表示跨节点事务与消息规则，“全局目录”表示 owner、sharer、状态和 epoch 组成的 committed directory。



2.2 控制路径与数据路径

Outer 协议对控制信息与数据传输采用统一事务身份进行关联：

- 控制路径：请求类型、权限状态、epoch、reqId、sharer/owner 信息和完成确认；
- 数据路径：远程读数据、脏数据回收、写回数据和授权返回；
- 完成路径：请求授权、失效确认、Clear 提交和最终权限可用。

数据来源由全局目录状态决定。若最新数据位于远程 owner，Home UBCC 控制器发起 Recall；若 Home 已有权威数据，则直接组织授权返回。该设计避免由节点内任意缓存副本替代全局 owner 语义。

2.3 组件关系

组件	所属域	主要职责
CPU Cache / HN-F	Inner 域	执行节点内缓存一致性和本地内存访问
EP-RNF	边界层	代表 Outer 域响应 HN-F snoop，并发起跨节点权限操作
EP-SNF	边界层	将节点内服务请求接入 Outer 数据路径

组件	所属域	主要职责
UBAdapter	边界层	完成 CHI 端点与 UBIO 之间的消息适配和事务关联
UBCC 控制器	Outer 域	维护全局目录，执行权限仲裁和事务收敛
ResidentDir	Outer 域	保存活跃跨节点目录元数据
H64 Backstore	Outer 域	保存冷目录元数据并支持换入换出

3. 核心组件设计

3.1 UBCC 控制器

UBCC 控制器承担 Outer 层的 Home-directory 与同址事务串行化职责。每条 64 B 缓存行由地址映射选择唯一 Home node/socket；该地址的 Home UBCC 控制器维护全局目录并决定权限授予、失效目标和提交顺序。sharer 按节点记录，Socket 用于 requester 身份和节点内路由。

全局目录采用 G_I/G_S/G_E/G_M MESI 类状态，记录 sharer 位图和 epoch；G_E/G_M 的 owner 由 one-hot sharer 推导。控制器将 committed state 与 intended state 分离：Grant 表示权限已经保留，匹配的 Clear 或本地升级完成事件到达后才提交新的全局目录状态。最新 64 B 数据可以位于 Home memory、远程 owner 或事务数据缓冲，数据位置不改变 Home UBCC 控制器的仲裁和提交职责。

控制器的主要功能包括：

1. 查询 committed directory 并选择权威数据源；
2. 为远程读、所有权迁移和共享转写者请求建立 intended state；
3. 生成精确 Recall 或 Invalidate 目标并收敛 Ack；
4. 管理同址主事务、等待请求、稳定重试身份和幂等完成记录；
5. 协调 ResidentDir 与 H64 Backstore 的目录元数据生命周期。

当前交付配置使用有界事务资源。表中的数量是实现上限，可随实现配置调整。

资源	当前上限	显式数据或位图存储	主要作用
活动 Outer 主事务	128	最多 $128 \times 64 \text{ B} = 8 \text{ KiB}$ Recall 数据	控制器并发事务准入和同址串行化
单地址 pending requester	32	包含在全局等待资源中	热点地址请求排队
pending requester 总数	256	最多 $256 \times 64 \text{ B} = 16 \text{ KiB}$ 写回数据	保留同址主事务等待期间的数据请求
ResidentDir waiter 总数	256	最多 $256 \times 64 \text{ B} = 16 \text{ KiB}$ 写回数据	保留 fill、替换或持久化等待期间的原操作
活动事务目标与 Ack 位图	每事务 6 个 64-bit 位图字段	$128 \times 48 \text{ B} = 6 \text{ KiB}$	记录失效目标、完成集合和升级目标

资源	当前上限	显式数据或位图存储	主要作用
H64 活动事务槽	128	最多 $128 \times 64 \text{ B} = 8 \text{ KiB}$ bucket RMW 快照	lookup、upsert 和 erase 准入
H64 持久化 waiter	64	最多 $64 \times 64 \text{ B} = 4 \text{ KiB}$ 写回数据	等待 H64 metadata 操作完成
H64 并行 bucket RMW	8	已包含在 128 个事务槽的 RMW 快照中	控制 metadata DRAM 并行修改
单一 H64 bucket waiter	8	事务槽索引和到达次序	串行化同 bucket 冲突

上述数据量只统计实现中明确保留的 64 B 数据和位图，预留上限合计为 58 KiB。其余控制字段包括地址、node/socket、状态、epoch、reqId、阶段和计时信息，随活动事务和等待请求数量线性增长；表中数值不将软件对象布局换算为芯片面积。

3.2 ResidentDir 与 H64 Backstore

ResidentDir 是全局目录的片上驻留层，采用 bit-packed set-associative 组织，并在每个 set 内使用 pseudo-LRU 选择候选条目。目录条目包含 valid、全局 MESI 状态、resident metadata dirty、fill/writeback/pinned 控制位、节点级 sharer 位图、24-bit epoch 和 tag。

当前基础配置的片上目录预算为 512 KiB：

配置	ResidentDir 数据区	Bloom	GroupIndex	ResidentDir 容量
naive	约 508 KiB	0	4 KiB	65,536 条
spill-noopt / optimized	约 448 KiB	60 KiB	4 KiB	57,344 条

片上预算满足：

$$B_{\text{onchip}} = B_{\text{resident}} + B_{\text{bloom}} + B_{\text{group-index}} + B_{\text{reserved}}$$

H64 Backstore 位于 metadata DRAM，保存从 ResidentDir 迁出的冷目录元数据，不保存缓存行数据。

每个 H64 bucket 占一个 64 B metadata line，由 4 B header 和 5 个 12 B slot 组成：

H64 结构	字段
4 B bucket header	format version、generation、live count、tombstone count
12 B slot	44-bit PA、2-bit MESI、2-bit slot state、16-bit sharer mask、24-bit epoch、8-bit integrity
slot state	EMPTY、LIVE、HASH_TOMBSTONE、RESERVED

H64 将地址映射到 256 个 routing group，再映射到组内 home bucket；lookup 从 home bucket 开始执行有界线性 probe，TOMBSTONE 不终止搜索，EMPTY 表示查找结束。upsert 更新匹配的 LIVE slot，或复用首个 TOMBSTONE/EMPTY slot；erase 将匹配项转为 TOMBSTONE。bucket 修改使用读一改一写序列，并以 generation、epoch 和 integrity 检测并发更新、过期写入和损坏。

当前配置提供 128 MiB metadata DRAM，并按 Socket 均分。单 Socket 配置下，每个 group 包含 8,191 个 bucket，H64 共提供 $256 \times 8,191 \times 5 = 10,484,480$ 个物理 slot。双 Socket 配置

在每个 Socket 上独立组织 4,095 个 bucket/group，总物理 slot 数为

$2 \times 256 \times 4,095 \times 5 = 10,483,200$ 。可用 live 容量还受目标装载率、哈希冲突和有界 probe 条件约束。

当前 12 B slot 使用 16-bit 节点级 sharer mask，可直接覆盖最多 16 个节点；Socket 只参与 requester 身份和节点内路由，不增加 sharer 位宽。16N1S 位于该编码范围内。若将 Socket 或 endpoint 作为独立全局 sharer，或扩展到 16 个以上节点，需要扩宽 slot 或采用间接、分层 sharer 编码。

等效追踪容量按缓存行地址计算 ResidentDir 有效条目与 H64 已持久化 LIVE 元数据去重并集，使固定片上预算优先服务热点目录，同时由 metadata DRAM 承担冷目录容量。

3.3 gem5 EP 边界架构

gem5 中的 EP 位于 Inner CHI 域与 Outer 一致性层之间。CPU cache 和 HN-F 继续执行节点内 CHI；EP-RNF、EP-SNF、EPBackend 和 UBAdapter 将节点内请求、snoop、数据与完成事件转换为 Outer 权限意图和事务结果。EP 不拥有全局目录或提交权，所有全局权限决策仍由目标地址的 Home UBCC 控制器完成。

EP-SNF 将节点内服务请求封装为 Outer 请求，并在数据与授权返回后生成 CHI 响应。EP-RNF 代表 Outer 层参与 HN-F 的本地 snoop，并为 Recall、Invalidate 和本地写升级发起相应的 ReadShared、ReadUnique 或 CleanUnique 子事务。UBAdapter 按 node/socket 路由消息，并保持 Outer epoch/reqId 与节点内事务完成之间的关联。

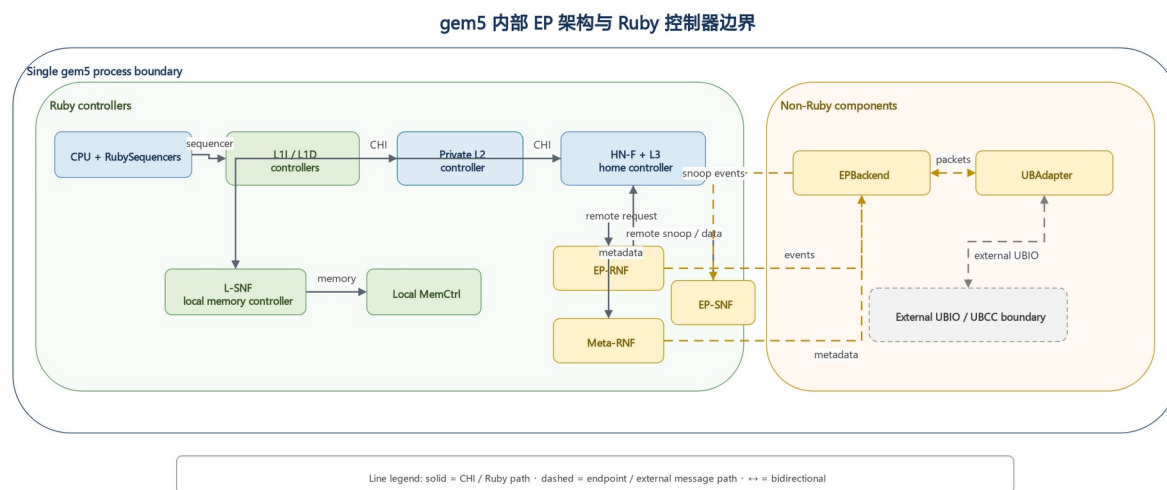


图 3-1 gem5 EP 架构

3.4 EP-RNF

EP-RNF 在节点内 CHI 域中代表跨节点一致性域。HN-F 对共享或独占缓存行发起 snoop 时，EP-RNF 根据当前 Outer 事务状态选择即时响应、返回 stale 结果或发起跨节点权限操作。

EP-RNF 的关键职责包括：

- 处理 SnpCleanInvalid、SnpUnique、SnpOnce 等 snoop；

- 将本地写升级转换为 Outer 权限请求；
- 为 Recall 发起节点内 ReadShared 或 ReadUnique；
- 在同址 CHI 事务与 snoop 并发时执行确定性仲裁。

3.5 EP-SNF

EP-SNF 将节点内无 snoop 服务请求连接到 Outer 数据路径。它接收节点内请求，完成地址和事务信息封装，并在 Home UBCC 控制器返回数据与授权后生成节点内响应。

3.6 UBAdapter

UBAdapter 提供稳定的消息适配边界，负责：

- 协议消息序列化与反序列化；
- 本地事务与跨节点事务身份关联；
- 请求发送、响应分发和回调完成；
- 对可恢复消息执行稳定 tuple 重试。

4. 全局一致性语义

4.1 目录状态

全局目录记录每条缓存行的全局权限关系，核心信息包括：

- 当前 owner；
- sharer 集合；
- 数据有效性与脏状态；
- committed epoch；
- 正在执行的权限事务。

全局状态采用单调 epoch 区分新旧事务。reqId 标识同一 epoch 内的具体请求，使重试、重复消息和延迟消息可以被准确识别。

4.2 请求与授权

一次跨节点操作由请求、仲裁、授权和提交组成：

1. requester 发送读或写权限请求；
2. Home UBCC 控制器查询已提交目录状态；
3. 必要时 Recall owner 或 Invalidate sharer；
4. Home UBCC 控制器返回数据和临时授权；
5. requester 完成本地操作后发送 Clear；
6. Home UBCC 控制器校验事务身份并提交新目录状态。

4.3 两阶段提交

Home UBCC 控制器将授权发送与目录提交分为两个阶段：

- 阶段 1（保留）：创建 outstanding，记录目标状态和事务身份，保持原已提交目录状态；
- 阶段 2（提交）：收到匹配的 Clear 后，提交目标状态并退役对应事务。

该语义保证 Grant 在途期间目录仍保持安全状态，并使重复请求能够返回同一授权结果。

4.4 幂等与过期消息处理

Outer 协议使用以下机制处理消息重复、延迟和重试：

- Ack 位图保证每个目标只贡献一次确认；
- epoch 和 reqId 拒绝过期事务；
- Clear tombstone 支持已完成事务的幂等确认；
- stable tuple 保证重试不改变事务身份；
- waiter 去重避免相同请求重复进入等待队列。

5. 关键协议路径

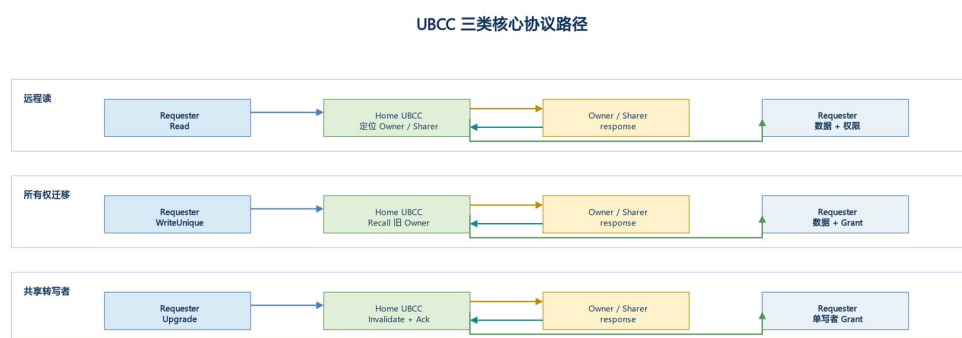


图 5-1 UBCC 核心协议路径

5.1 远程读

远程读的目标是定位权威数据并将共享权限返回 requester：

1. requester 的节点内 miss 经 EP-SNF 发送到 Home UBCC 控制器；
2. Home UBCC 控制器查询 owner 和 sharer 状态；
3. 若远程 owner 持有最新数据，Home UBCC 控制器发起 Recall；
4. owner 节点经 EP-RNF 读取节点内权威副本并返回数据；
5. Home UBCC 控制器更新共享关系并向 requester 返回数据和授权。

5.2 所有权迁移

所有权迁移用于将写权限和最新数据从旧 owner 转移到新 requester：

1. 新写者向 Home UBCC 控制器请求独占或修改权限；
2. Home UBCC 控制器定位旧 owner；
3. 旧 owner 完成本地降级或失效，并返回最新数据；
4. Home UBCC 控制器完成权限重配置；
5. 新写者获得数据和单一写权限。

该路径的优势来自全局目录对最新数据位置的直接定位，以及权限释放与新授权之间的统一事务管理。

5.3 共享转写者

共享转写者路径用于将多个共享副本收敛为单一写者：

1. requester 发起写权限请求；
2. Home UBCC 控制器确定并保持本次事务的有效 sharer 目标集合；
3. Home UBCC 控制器向目标节点发送 Invalidate；
4. 每个目标完成本地失效并返回 Ack；
5. Home UBCC 控制器在 Ack 集合收敛后向 requester 授权；
6. Clear 到达后提交新的 owner 状态。

5.4 写回与逐出

节点逐出脏数据时，Home UBCC 控制器根据 committed directory 和事务 epoch 校验写回来源。有效写回

可作为 Recall 的权威数据返回；重复或过期写回不会重复提交目录状态。

5.5 目录换入与换出

ResidentDir 以 set 为单位管理容量。目标 set 无空闲位置时，Home UBCC 控制器从 pseudo-LRU 位置开始选择候选条目，并跳过当前访问地址和 pinned 条目。以下状态会使条目保持 pinned：

- 该地址存在活动 Outer 主事务；
- 请求正在等待目录换入或 metadata writeback；
- 条目正在执行 H64 upsert、erase 或数据持久化；
- waiter 的完成依赖该条目继续存在。

换出按照目录状态和持久化状态分类处理：

1. H64 已保存相同 epoch 的有效目录副本时，未修改的驻留条目可以直接释放 ResidentDir 位置；
2. 修改后的有效条目先执行 H64 upsert，写入 MESI、sharer 和 epoch，收到持久化确认后释放；
3. 已转为 G_I 且 H64 仍有旧记录的条目执行 erase，确认后释放；
4. set 内全部 way 均被 pin 时，新请求进入容量 waiter，不覆盖任何仍有全局目录意义的条目。

ResidentDir miss 的换入路径先检查对应分组 Bloom。可信的 negative 表示 H64 中不存在该地址，Home UBCC 控制器可直接建立新的 G_I 条目；positive 或正在重建的 Bloom slice 触发 H64 lookup。lookup Found 时恢复 MESI、sharer 和 epoch，NotFound 时建立 G_I 条目。H64 暂时无法准入时，placeholder、原请求类型、node/socket、epoch、reqId 和数据负载保持不变，待资源可用后重试。换入完成后，Home UBCC 控制器解除 fill 状态，重新检查全局目录和活动事务，再按原事务身份重放 waiter。过期 epoch、损坏 bucket 和耗尽的 probe 路径分别进入对应错误处理，不转换为新的空目录状态。

6. 并发仲裁与活性机制

6.1 同址事务串行化

Home UBCC 控制器对同一缓存行保持单一主事务。并发请求根据 committed state、outstanding stage 和

请求类型进入以下处理之一：

- 立即服务；
- 进入 waiter 队列；
- 返回 BUSY 并按稳定事务身份重试；
- 合并到正在进行的 Recall 或 Invalidate 流程。

6.2 动态失效目标

失效目标以发送时刻的 committed directory 为基础计算，并扣除已完成降级或已确认的目标。该机制使 partial Ack 重试只覆盖尚未确认的节点，避免重复扩大失效范围。

6.3 EP-RNF snoop 仲裁

EP-RNF 对同址 CHI 事务和 snoop 采用分类仲裁：

- active Recall 优先完成数据回收；
- 可安全即时响应的 snoop 直接完成；
- 与写权限冲突的 snoop 返回 stale 结果，使发起者按全局顺序重试；
- 不符合路由约束的组合进入协议错误处理。

6.4 waiter 精确退役与重放

Clear 成功提交后，Home UBCC 控制器按 (PA, node, socket, reqId) 精确退役已经完成的 Read waiter，保留其他 requester、其他事务身份和其他操作类型的 waiter，再安全重放剩余请求。

6.5 可恢复消息重试

Clear、Upgrade、Invalidate 和 Recall 路径均保存原事务身份。发生可恢复消息丢失时，协议重发相同 tuple，并由接收方按 epoch、reqId 和 Ack 状态执行幂等处理。

7. Outer 协议性能分析

7.1 比较范围与评价维度

Outer 协议的性能由两个正交维度共同决定：稳定状态集合决定协议能够直接表达哪些副本和数据责任，Outer 组织方式决定目录权威、数据路径和消息承载如何分布。MESI、MOESI 或 MESIF 可以分别与专用 Home-directory、直接数据转发或 Outer CHI 组合，因此不能只按状态数量判断完整方案。

本节使用以下评价量：K 表示 requester 可见完成路径上的串行跨节点消息段数，M 表示跨节点单播消息总数，D 表示完整 64 B 数据线的跨节点遍历次数，S 表示需要失效的实际

sharer 数，F 表示同时接收 Recall 或 Invalidate 的目标数。

状态、事务承载与全局权威的职责分离

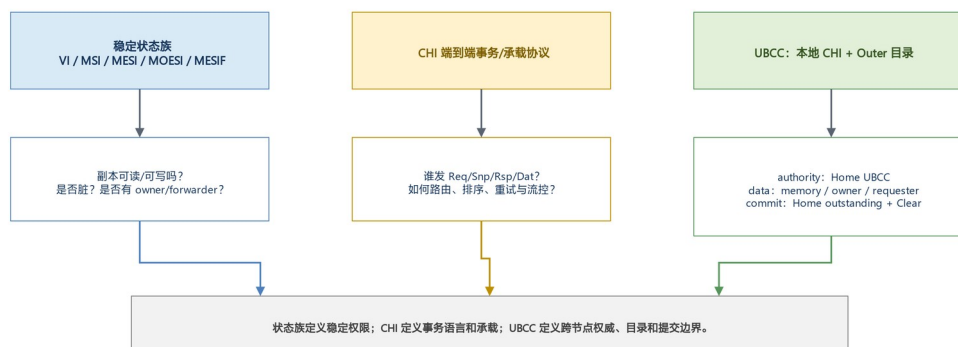


图 7-1 状态集合、事务承载与全局权威

半定量时延可表示为：

$$T_{\text{visible}} \approx K \times \tau_{\text{link}} + T_{\text{dir}} + T_{\text{local}} + T_{\text{queue}} + T_{\text{fanout_tail}}$$

该表达用于比较依赖关系和资源趋势；实际结果还取决于目录命中率、互连带宽、控制器并行度和具体实现时序。

7.2 状态集合的性能影响

稳定状态描述事务收敛后的副本关系，瞬态状态则承接数据返回、权限变更和确认之间的依赖。

以下分别讨论五种状态集合；其性能收益以相应访问模式出现为前提，不等同于状态数量的排序。

7.2.1 VI：简洁副本状态与外部权限管理

VI 用有效与无效表示本地副本是否可用，适合副本管理简单、权限责任由外部机制承担的组织。有效位本身不区分唯一写者、多个读者或脏数据责任，因此跨节点写入仍需借助目录、集中仲裁或探测来证明其他副本不再可写。VI 不意味着每次访问都广播，也不意味着可以省去一致性仲裁。其局部状态判断路径短，稳定状态至少需要 1 bit；但远程 miss 的 latency 取决于附加机制如何找到数据源。若采用探测，traffic 与接收端处理会随候选节点数增加，限制持续 throughput；若配合精确目录，则仍需保存节点级 sharer 和事务状态。因而 VI 的适用性应按整个权限管理组织的 storage 和消息成本评估，而非只比较一个有效位。

7.2.2 MSI：显式写者与共享集合

MSI 以 M 表示持有修改责任的单一 owner，以 S 表示只读共享副本，以 I 表示无效。目录式组织可以据此向 owner 回收最新数据，或向实际 sharer 发起失效，避免向无关节点查询。对确实存在多个读者的缓存行，这种表达已经覆盖共享读取和单写者切换的主要稳定关系。单一干净副本在 MSI 中仍表现为 S，首次写入需要完成 S→M 权限升级，即使实际上没有其他 sharer，也要由全局仲裁确认。私有读后写场景因此增加控制往返、Home 事务占用和排队机会；持续共享场景则不一定因缺少 E 而增加相同成本。MSI 至少使用 2 bit 稳定状态，精确目录仍需位图和身份字段，适合共享访问占主导、私有干净升级优化价值较低的场景。

7.2.3 MESI：利用干净独占副本缩短私有写升级

MESI 增加 E，表示一个节点持有唯一的干净副本。全局独占关系成立时，该节点可按本地一致性规则完成 $E \rightarrow M$ ，无需重新执行共享副本失效路径。这对初始化后由单个节点使用的私有数据、先读后写的数据结构尤其有利：写入 latency 降低，同时减少跨节点升级消息和 Home 事务槽占用。E 的收益来自“已知唯一”，而不是绕过全局权限管理。其他节点请求该缓存行时，仍需由 Home UBCC 控制器协调降级或迁移；高共享度下 E 停留时间较短，其 throughput 收益也随之减少。MESI 与 MSI 均可用 2 bit 编码，新增成本主要是 E 相关转换和本地写升级的状态衔接，而非稳定状态位宽。当前 UBCC 的 MESI 类全局目录采用这一权限表达，并由第 4 章的授权与提交机制维护全局关系。

7.2.4 MOESI：保留脏共享数据责任

MOESI 增加 O，使一个节点在其他节点持有只读副本时继续承担脏数据责任。对于一个节点产生数据、多个节点随后读取的模式，O 持有者可以作为后续读取的数据源，不必先使 Home memory 成为最新副本再服务每个读者。收益集中在可避免的写回和内存访问，而非所有远程读取。O 不规定数据必须直接到 requester：Home 中转仍会占用两段数据带宽，直接转发才可能进一步减少数据遍历。长期保留 O 可以降低 Home memory traffic，但也会使供数节点承担热点带宽；O 逐出、责任迁移和共享转写者都需要额外协调，可能增加事务占用和尾延迟。稳定状态至少需要 3 bit，瞬态 storage 还要表达脏共享责任的释放与接续。该状态集合适合脏共享复用充分的访问模式，在本章作为架构比较选项，当前 UBCC 全局目录采用 MESI 类状态。

7.2.5 MESIF：为干净共享读取指定响应者

MESIF 增加 F，在多个干净共享副本中指定一个转发响应者。新读者到达时，目录和消息规则可以选择该响应者供数，减少多个副本同时响应或重新选择数据源的工作。与 O 不同，F 表达响应责任而非脏数据责任，适合读多写少、共享副本能够持续复用的数据。

若 cache-to-cache 路径优于 Home memory 路径，F 可降低读 latency 和内存 traffic；若响应者较远或已饱和，指定 F 并不自动改善 throughput。F 逐出、迁移或被失效时，需要保持响应者身份与目录一致，写密集模式会反复支付这些维护消息。至少 3 bit 的稳定状态只是 storage 增量的一部分，还需相应瞬态和完成跟踪。MESIF 同样是本章的比较选项，不能由节点内 CHI 供数能力推导为当前 Outer 已具备 F 语义。

状态位本身通常不是目录存储的主要部分。精确目录还需要 sharer 位图、tag、epoch 和控制位；事务执行期间还需要 requester、目标位图、Ack 位图和可选的 64 B 数据缓冲。下表归纳各状态集合的主要取舍，实际资源应结合第 7.7 节的目录与瞬态开销评估。

状态集合	直接表达能力	Latency 影响	Throughput、Traffic 与 Storage 影响
VI	只区分有效与无效	需要额外机制定位写者、共享者和脏数据源	状态处理简单；精确失效和数据源选择依赖附加目录或探测；至少 1 bit 状态
MSI	表达单一 Modified owner 与 Shared 集合	单一干净副本写入仍经过 S→M 升级	可按 owner/sharer 定位目标；至少 2 bit 状态
MESI	增加 Exclusive	单一干净持有者可本地 E→M，减少一次全局升级路径	降低私有干净数据的控制流量和 Home 事务占用；至少 2 bit 状态
MOESI	增加 Owned	dirty shared read 可由 O 持有者供数，减少回写后再读路径	降低部分 Home memory 数据流量，增加 O 回收、转发和逐出事务；至少 3 bit 状态
MESIF	增加 Forward	多 sharer 读取时由 F 提供确定的 cache-to-cache 数据源	减少响应者选择和部分 Home/memory 数据流量，增加 forwarder 选举、迁移和失效流量；至少 3 bit 状态

7.3 Outer 组织方式

Outer 组织决定消息如何到达仲裁点、数据如何跨节点移动，以及压力由哪些端口与事务资源承担。这一比较轴独立于第 7.2 节的状态集合，图 7-1 展示了两者与全局权威的关系。

7.3.1 专用 Home-directory 与 Home 数据中转

每条地址由唯一 Home UBCC 控制器管理目录和提交。远程 owner 返回最新数据后，Home 再将数据与授权组织为面向 requester 的响应，使数据回收和权限推进在同一事务上下文内关联。这也是当前 UBCC 的 Outer 组织，节点内 CHI 继续负责本地缓存层级的一致性。

该方式适合需要独立全局目录资源、精确目标选择和分层元数据容量的系统。Home memory 本身为最新数据源时，数据可以直接从 Home 返回；远程 owner 为最新数据源时，中转增加完整数据线的链路遍历和 Home 数据端口负载。吞吐因此同时受目录并行度、事务槽占用及数据带宽制约。storage 除稳定目录外还包括 Recall 数据缓冲和等待状态，第 3.1 节给出了当前配置的资源规模，不能只按目录条目数衡量整个控制器。

7.3.2 专用 Home-directory 与直接数据转发

直接数据转发保留 Home 的权限判定和提交职责，只改变经授权的数据分支：Home 指定合法 source 与 target 后，owner 将数据直接发给 requester。对 Home、owner 和 requester 分离的远程供数场景，这可省去一次完整数据线中转，降低 Home 数据带宽压力，并缩短数据分支。requester 仍需同时取得数据与有效权限，旧 owner 的释放及失效确认也仍属于完成条件。

因此改善 latency 的幅度取决于权限分支是否更慢；对 Home memory 直接供数的访问则没有相同收益。throughput 的瓶颈可能从 Home 转移至供数节点或互连出口。事务 storage 需要

关联授权、source/target 和数据完成，处理两个分支不同的到达次序。这种组织适合远程 owner 数据流量成为主要约束的系统，是与当前 Home 中转组织相区分的架构选项。

7.3.3 Outer CHI：标准事务承载与跨域衔接

Outer CHI 将跨节点事务组织为 CHI 的 RN/HN/SN 角色以及 Req/Rsp/Snp/Dat 通道，由相应的全局 Home 和目录规则管理一致性。对已有跨 die/Socket CHI fabric 或第三方 agent 互操作需求的系统，标准事务、数据路径和 credit 流控可以复用既有互连能力；具体 DMT/DCT 路径仍取决于所选配置和 agent 支持，协议名称本身不保证每条事务都采用最短数据路径。

独立通道和流控有助于组织并行事务，但 credit 等待、ID 占用和 bridge 背压仍会增加 latency 并限制 throughput。traffic 由 snoop 目标和实际数据路由决定，不能仅以采用 CHI 推断其低于专用目录消息。storage 需计入 agent 状态、TxnID/DBID 关联、通道缓冲及完成关系。当前 UBCC 在节点内复用 gem5 CHI，Outer 使用专用目录消息；图 7-4 对比的是这一边界与 Outer CHI 组织所需资源，而非将节点内能力等同于跨节点实现。

7.3.4 广播或探测式 Outer：以接收端工作换取较小目录

在不保存完整精确 sharer 或只保存有限提示时，请求可以向候选节点广播或探测，再根据响应确定数据源和权限关系。这降低稳定目录位图的存储要求，适合节点数较少、探测域受控或目录 SRAM 预算极紧的组织。有限提示还可以缩小候选范围，但需要相应规则保证遗漏节点不会保留冲突权限。

并行发送使探测段数不必随节点数线性增长，然而 traffic、接收端过滤和响应收敛工作会增长。在共享稀疏时，许多节点处理的是与自身无关的请求；在高负载时，这些工作增加链路和队列竞争，抬高 latency 尾部并压低有效 throughput。省下的稳定 storage 还应与网络缓冲、响应集合及接收端瞬态资源一起计算，尤其不能把广播包的单次注入等同于全网只有一份传输成本。

下表将数据组织与权威位置分开归纳；采用何种稳定状态均需配套完整的授权和完成规则。

Outer 组织	目录与提交权威	数据路径	主要性能特征
专用 Home-directory + Home 中转	每地址唯一 Home UBCC 控制器	owner 数据返回 Home，再发送 requester	权威和完成顺序清晰；Home 承担目录访问和数据带宽
专用 Home-directory + 直接数据转发	Home 决定合法 source/target 并提交	owner 在 Home 授权后直接发送 requester	可将数据遍历从两次降为一次；控制和提交仍经过 Home
Outer CHI	全局 CHI Home/目录和 agent 规则	由 CHI data path、DMT/DCT 等机制组织	标准互操作和硬件流控能力强；增加通道、credit、ID 和 bridge 状态
广播或探测式 Outer	不保存完整精确 sharer，或只保存有限提示	向候选节点广播或探测数据和权限	稳定目录较小；流量和接收端处理随节点数增长

7.4 远程读与所有权迁移

比较数据路径前需要区分目录权威、数据位置和提交权威，再按最新数据所在位置分析路径。

以下路径计数沿用第 7.1 节的 K 与 D ，用于表示依赖与遍历，不包含所有实现细节或重试。

7.4.1 Coherence authority: 决定合法权限与目标

Home UBCC 控制器依据 committed directory 判定当前 owner、节点级 sharer 和可授予权限。即使某个节点已经持有可供读取的数据，也需要这一判定来确保供数身份和新副本权限有效。对远程读，这决定向谁 Recall；对写请求，这决定必须收敛哪些共享副本。目录命中和目标选择位于权限关键路径，因此 ResidentDir 命中率影响 latency，查询端口和并发事务资源影响 throughput。精确节点级位图用稳定 storage 换取较少的无关探测 traffic；Socket 用于 requester 身份与节点内路由，并不扩大 sharer 数。这一权威职责与数据是否经过 Home 无关，直接转发同样需要有效的目录判定。

7.4.2 Data location: 决定完整数据线的移动成本

最新 64 B 数据可能位于 Home memory、远程 owner 或事务数据缓冲。Recall 与写回可以改变数据位置，但不会把全局目录权威随数据一起移交。区分这两个概念后，才能判断一次优化是在减少内存访问、缩短互连路径，还是只改变控制消息的组织。数据位置直接影响 D 、供数 latency 与端口 traffic；热点 owner 的出口和 Home 中转端口都可能成为 throughput 限制。事务缓冲还需要保存回收中的数据，直到对应权限步骤能够推进。因此 storage 分析应将目录元数据与 64 B 数据缓冲分开：H64 保存冷目录元数据，不是保存这些缓存行数据的另一层数据缓存。

7.4.3 Serialization/commit authority: 决定同址可见顺序

Home UBCC 控制器以同址主事务协调冲突请求，将 intended state 与 committed state 分开。Grant 保留权限，匹配的 Clear 到达后提交新目录状态；数据提前抵达并不使下一笔冲突请求自动获得新的权限。该机制把供数完成与全局关系更新联系起来，同时保持两者的职责独立。同址热点的 throughput 受主事务占用时间限制，不同地址则可利用多个事务槽并行推进。等待请求和完成记录增加瞬态 storage，Clear 等控制消息增加 traffic，但提供了明确的提交顺序和幂等完成依据。第 7.1 节的 requester 可见 latency 与事务退役时间应分别观察：数据分支变短可以加快读者取得数据，却未必同比缩短同址排队和控制器资源占用。

7.4.4 Home memory 最新: 直接由 Home 供数

当 Home memory 是权威数据源时，请求与数据返回形成 requester→Home→requester，约为 $K \approx 2$, $D=1$ 。这里的数据起点已经是 Home，增加远程转发角色不会再减少一次数据中转。E、O、F 影响副本责任表达，但不凭空改变本次数据的位置。这一场景的 latency 主要由请求往返、目录和内存访问决定；throughput 取决于 Home memory 与返回链路的服务能力。数据 traffic 为一次完整数据线遍历，事务仍需授权与完成状态，但不需要为远程 owner Recall 额外建立供数分支。它适合作为比较远程供数组织的基准场景，尤其应与 owner 最新场景分开统计，避免把直接转发收益套用到所有远程 miss。

7.4.5 远程 clean holder/F 供数：路径与响应责任共同决定收益

若组织允许并选择远程干净持有者供数，Home 中转路径是 requester→Home→holder→Home→requester，约为 $K \approx 4, D=2$ ；Home 授权后直接供数的数据分支约为 $K \approx 3, D=1$ 。

MESIF 的 F 可指定唯一干净响应者，其他状态集合若采用 clean holder 供数也需要明确的数据源选择规则。该路径是组织方式比较，不表示当前 UBCC 已采用 F 或任意干净副本转发。当远程缓存访问与链路的组合成本低于 Home memory 访问时，cache-to-cache 可改善 latency；若 holder 较远或繁忙，目录选择和额外节点访问也可能抵消收益。直接供数降低 Home 数据 traffic，却把服务压力转移到 holder，throughput 应连同该节点出口一起评估。storage 方面，F 责任维护与直接传输完成跟踪是两类独立成本，不能把前者视为自动提供后者。

7.4.6 远程 dirty owner 最新：先满足数据与权限依赖

远程 owner 持有最新脏数据时，Home memory 不能替代该数据源。当前 Home 中转通过 requester→Home→owner→Home→requester 回收并返回数据，约为 $K \approx 4, D=2$ 。直接转发组织在 Home Recall/授权后由 owner 向 requester 供数，数据分支约为 $K \approx 3, D=1$ 。MOESI 的 O 可以在只读共享形成后保留脏数据责任，而 MESI 类目录按其降级规则完成共享转换。所有权迁移还要求旧 owner 释放权限、其他有效副本完成必要失效。数据可能先到，权限也可能先就绪，requester 可见完成受较慢分支约束：

$T_{\text{visible}} = \max(T_{\text{data}}, T_{\text{authority}})$ 。这解释了为什么减少数据

遍历能够降低 traffic，却不一定同比降低写迁移 latency。热点 owner 供数、失效尾部和主事务占用共同决定 throughput；Recall 数据缓冲、目标集合及两个分支的完成关联构成瞬态 storage。该场景最能体现数据路径优化的价值，也最需要保持权限依赖的完整分析。

图 7-2 对比上述 Home 中转和直接转发的数据分支；下列两表分别总结职责与路径。

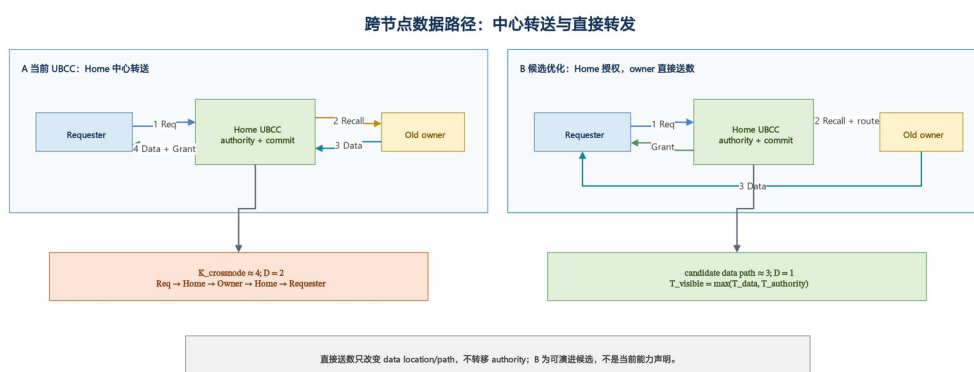


图 7-2 Home 中转与直接数据转发

概念	回答的问题	当前 UBCC 中的位置
coherence authority	谁判定 owner、sharer 与可授予权限？	Home UBCC 控制器的 committed directory
data location	最新 64 B 数据在哪里？	Home memory、远程 owner 或事务数据缓冲

概念	回答的问题	当前 UBCC 中的位置
serialization/commit authority	谁定序并提交同址状态？	Home UBCC 控制器主事务；匹配 Clear 后提交

场景	Home 中转	直接数据转发	状态集合影响
Home memory 最新	requester→Home→requester, $K \approx 2, D=1$	数据已在 Home, 无额外收益	E/F/O 不改变该数据位置
选择远程 clean holder/F 供数	requester→Home→holder→Home→requester, $K \approx 4, D=2$	Home 授权后 holder→requester, 数据分支约 $K \approx 3, D=1$	MESIF 的 F 可指定唯一 clean cache-to-cache 响应者
远程 dirty owner 最新	requester→Home→owner→Home→requester, $K \approx 4, D=2$	Home Recall/授权后 owner→requester, 数据分支约 $K \approx 3, D=1$	MOESI 的 O 可保留 dirty shared 数据源

7.5 Shared-to-Writer 与失效扇出

设除 requester 外需要失效的 sharer 数为 S 。精确目录只向这 S 个节点发送 Invalidate；请求、并行 Invalidate、并行 Ack 和 Grant 构成约四层依赖：

$$\begin{aligned}
 K &\approx 4 \\
 M &\approx 2S + 2 \\
 F &= S
 \end{aligned}$$

若将 requester 的 Clear 计入，则消息数约为 $2S+3$ 。MESI、MOESI 和 MESIF 在建立单写者时都需要收敛其他有效副本，差别主要在于写入前的数据来源，以及是否存在 O/F 生命周期。广播或探测式协议在不知道精确 sharer 时需要向 $N-1$ 个节点发送请求，控制消息约为 $2(N-1)+2$ 。当实际 S 远小于 $N-1$ 时，精确目录显著减少链路流量和接收端处理；当副本接近全共享时，两者的最坏扇出趋近。

7.6 吞吐、流量与排队

持续吞吐取决于不同地址之间的并行度和单地址事务占用时间。Home-directory 方案在同一地址上保持串行化，不同地址可以并行使用目录和事务槽。以下因素会增加排队时间并降低吞吐：

- 长时间占用 Home 事务槽的 Recall、Invalidate 和目录换入；
- 大 sharer 集合产生的 Ack 尾部；
- Home 中转完整数据线形成的数据端口带宽压力；
- H64 lookup、upsert 或 erase 的 metadata DRAM 排队；
- Outer CHI 中 Req/Rsp/Snp/Dat credit 和 bridge backpressure；
- 广播或探测在非目标节点产生的接收与过滤工作。

状态集合对吞吐的影响取决于工作负载。MESI 的 E 状态缩短私有写升级；MOESI 的 O 状态减少脏共享回写和 Home 数据流量；MESIF 的 F 状态减少共享读取的响应选择。额外状态只有在减少的路径占用大于新增状态维护和恢复成本时才提高总体吞吐。

对 N 个节点， $N=2/8/16$ 时最坏 shared-to-writer 目标数分别为 1/7/15，精确目录下 $M \approx 2S+2$ 分别为 4/16/32。低共享度下，广播方案仍支付接近 $2(N-1)$ 的控制消息，精确目录则使流量随实际共享度 s 增长。

7.7 目录与瞬态存储开销

对 N 个可缓存节点，精确全位图目录的概念存储量可写为：

$$B_{\text{dir}}(N) = N + b_{\text{state}} + b_{\text{epoch}} + b_{\text{ctrl}} + b_{\text{tag}}$$

当前实现由 MESI 状态和 one-hot sharer 推导 owner，因此不单独保存 owner 编码。 b_{ctrl} 包括 valid、dirty、驻留和持久化控制位。仅 sharer bitmap 对 1 Mi 条目录记录的原始存储为： $N=2$ 时 0.25 MiB， $N=8$ 时 1 MiB， $N=16$ 时 2 MiB。VI/MSI/MESI 的状态均可在 2 bit 内编码，MOESI/MESI 需要至少 3 bit；相对 sharer、tag 和 epoch，增加 1 bit 状态的容量增量较小。

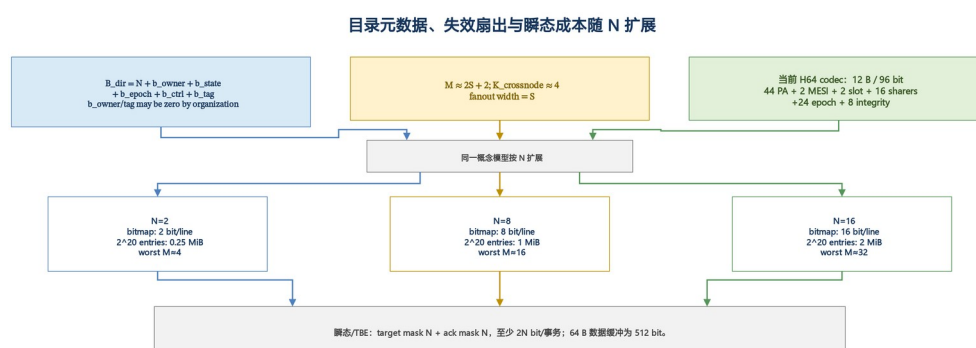


图 7-3 目录元数据、失效扇出与瞬态成本

瞬态资源通常大于稳定状态差异。一个 shared-to-writer 事务至少保存 requester、阶段、epoch、reqId、目标位图和 Ack 位图；需要回收数据时还保存一条 64 B 缓存行。Outer CHI 还需要 TxnID/DBID、四通道完成关系和 credit 状态；直接数据转发需要 source/target 授权与数据完成证明；广播方案减少稳定目录位图，却增加网络和接收端瞬态工作。

7.8 状态集合与组织方式的组合选择

组合选择应先识别限制性能的访问模式与资源，再决定是否增加状态责任或改变数据组织。前者针对副本关系，后者针对消息与数据承载；两者可以组合，但收益和成本需要分别核算。

7.8.1 MESI、专用 Home-directory 与 Home 中转

这一组合利用 E 缩短私有干净数据的写升级，通过精确节点级 sharer 限制失效范围，并将目录与同址提交集中在地址对应的 Home UBCC 控制器。它适合同时刻重视全局权限精度、节点内资源隔离和目录容量扩展的系统，也是当前 UBCC 所采用的组合。

私有读后写访问减少全局升级占用，稀疏共享减少无效控制 traffic；远程脏数据则承担 Home 中转的 latency 与数据带宽。throughput 需兼顾目录并行度和 Home 端口，而不是只提高事务槽数量。storage 由 2-bit 稳定状态、精确位图、分层元数据和专用事务资源组成。ResidentDir 与 H64 的价值在于按冷热度分配元数据容量，不改变最新缓存行数据的供数责任。

7.8.2 MESI/MOESI、专用 Home-directory 与直接数据

当 owner 供数占比高且 Home 数据端口成为主要瓶颈时，可以保持专用目录权威，将数据分支改为授权后的 source→requester。MESI 已可与该组织组合；若脏共享读取还能持续复用数据，MOESI 的 O 则进一步减少回写需求。因此“直接数据”和“Owned”分别优化路径与数据责任，不能视为必须同时采用的一项能力。

这类组合减少完整数据线中转，有助于数据受限的 throughput，也可能缩短读或 handoff 的数据 latency；权限收敛占主导时，可见完成收益受限。storage 需增加传输授权与完成跟踪，采用 O 时还需管理脏共享生命周期。适用判断应同时检查供数节点出口、互连竞争和共享模式，避免以 Home traffic 下降代替整个系统收益。该组合在此用于架构取舍分析。

7.8.3 MOESI/MESIF 与 Outer CHI

已有跨 die/Socket CHI fabric 的系统，可以将 O 的脏共享责任或 F 的干净响应责任映射到相应 agent、目录和事务规则中，复用标准 snoop 与 data 通道组织。O 与 F 解决不同共享模式，应由工作负载和 agent 能力选择；使用 CHI 本身不等于实现任意完整状态集合。

其 latency 与 throughput 收益取决于实际转发路径、通道并行度和 credit 供给，traffic 取决于 snoop 过滤与数据源选择。storage 则应包含 agent、channel、ID、credit 和跨域 bridge 状态，而不仅是 3-bit 稳定状态。当互操作或现有 fabric 复用具有明确价值时，这些成本才有对应的系统收益。图 7-4 保持节点内 CHI 与 Outer 组织的职责区分，当前 UBCC 的 Outer 仍由专用消息和 Home UBCC 控制器管理。

7.8.4 VI/MSI 与广播或探测

小规模系统可用较简洁的稳定状态结合受控探测域来管理副本，以较小的稳定目录换取节点参与权限查询。VI 需要外部机制补足写者与脏数据责任，MSI 则已表达 M 与 S；二者均需完整的仲裁、失效和完成规则，不能由局部状态简单推导全局事务也同样简单。

节点数少且互连负载较低时，并行探测的 latency 可以受到控制；规模扩大后，候选节点工作、响应尾部和网络排队会限制 throughput。尤其当实际 sharer 很少时，traffic 仍可能接近全节点范围。storage 评估应同时计入接收端与网络瞬态缓冲。该组合适用于探测域足够小或稳定目录预算极紧的条件；节点扩展和稀疏共享则更能体现精确目录的价值。

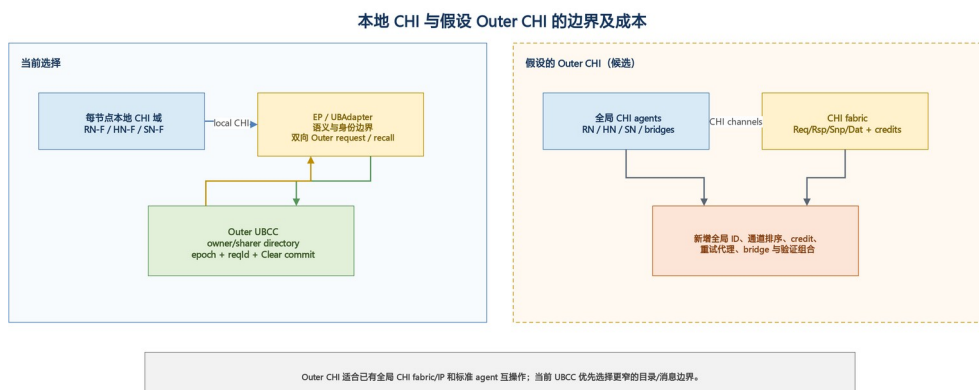


图 7-4 本地 CHI 与 Outer 组织边界

下表汇总组合取舍。当前组合以精确 sharer、稳定事务身份和独立分层目录兼顾权限、流量与容量；其他组合分别对应 owner 数据带宽、脏共享复用、互操作或目录预算等不同主导约束。

组合	Latency/Throughput 特征	Traffic 特征	Storage 特征	适用条件
MESI + 专用 Home-directory + Home 中转	私有 E→M 路径紧凑；脏远程数据经过 Home	精确 sharer 减少无效失效；Home 数据带宽较高	2-bit 状态、精确位图和专用事务状态	重视权威边界、资源隔离和容量扩展
MESI/MOESI + 专用 Home-directory + 直接 数据	dirty read/handoff 数据路径缩短	减少一次完整数据线路中转	增加直接传输授权和完成状态	owner 数据流量成为主要瓶颈
MOESI/MESIF + Outer CHI	可复用标准转发和数据通道机制	fabric 提供标准 snoop/data 路径	增加 agent、channel、credit 和 ID 状态	已有跨 die/Socket CHI fabric 或第三方互操作需求
VI/MSI + 广播/探测	控制简单，规模增大时排队和尾部上升	低共享度时仍接近全节点流量	稳定目录较小，网络瞬态开销较大	小规模或目录存储极受限系统

7.9 综合结论

状态集合主要决定协议能否直接表达 Exclusive、Owned 或 Forward 等数据责任；Outer 组织方式主要决定目录访问、数据移动、流控和提交如何分布。理论比较表明：

1. MESI 相对 MSI 的主要收益是私有干净数据的本地 E→M 升级；
2. MOESI 和 MESIF 分别面向脏共享数据源和高读共享响应者优化，以更多瞬态状态换取更少数据回写或响应选择；
3. 精确 Home-directory 在低共享度下减少失效流量，并提供稳定的同址提交点；
4. 直接数据转发降低 Home 数据带宽和完整数据线遍历次数，仍保留 Home 权限分支；
5. Outer CHI 提供标准 agent 与硬件流控能力，同时增加 channel、credit、ID 和 bridge 资源；
6. 广播或探测降低稳定目录存储，在节点规模和稀疏共享度增长时付出更高控制流量。

当前方案选择 MESI 类状态、精确 Home-directory、分层目录和专用 Outer 消息，以满足当前交付对权限精度、容量扩展、可恢复重试和节点内 CHI 复用的共同要求。

8. 集成边界

8.1 模块级交付边界

模块	对外职责	稳定边界
gem5 节点模型	节点内 CHI 与缓存层级	CHI 请求、snoop、数据与完成事件
EP 边界层	Inner 与 Outer 语义转换	权限意图、数据和事务完成

模块	对外职责	稳定边界
UBIO	Home UBCC 控制器、ResidentDir 与 H64 Backstore	Outer 请求、响应、确认与目录生命周期
framework	公共消息、端口和仿真器适配	事务身份、路由身份和数据负载

8.2 parallel_test_v2 填写模板

以下内容仅为未验证模板，不构成已交付功能声明；实际信息在集成阶段填写。

必要字段	填写值
测试集合选择方式	[待填写]
并行任务数	[待填写]
单项超时	[待填写]
结果判定与汇总位置	[待填写]

9. 总结

UBCC 方案以独立全局目录为核心，在保持节点内 CHI 一致性边界的同时，提供跨节点数据定位、权限仲裁、目录容量扩展和可恢复消息处理。该架构兼顾协议清晰度、容量效率、目标选择精度和多拓扑扩展能力，并已形成可集成到 ubsim 的模块化实现。

附录 A 消息与状态速查表

A.1 主要消息

消息类别	代表消息	作用
请求	ReadReq、UpgradeReq、RecallReq、InvalidateReq	发起数据或权限操作
响应	ReadResp、UpgradeResp、RecallResp	返回数据、目标集合或接受状态
确认	ClearReq、ClearResp、InvalidateAck、UpgradeAckNotify	确认本地完成或全局收敛
生命周期	Writeback、Evict	归还数据或释放目录关系

A.2 关键目录概念

概念	说明
committed state	已对后续请求可见的全局目录状态
intended state	当前授权完成后准备提交的目标状态
outstanding	正在执行的同址主事务
waiter	等待同址事务或目录换入完成的请求
tombstone	已完成事务的短期幂等记录

附录 B EP-RNF 仲裁规则

活跃事务	invalidating snoop	SnpOnce	处理原则
CleanUnique	stale	stale	保持全局写顺序，发起者重试
ReadUnique	stale	stale	优先完成 Recall 数据回收
ReadShared	stale	immediate data	允许只读数据即时返回
无冲突事务	immediate	immediate	按 CHI 正常响应

附录 C 术语表

术语	说明	术语	说明
UBCC 方案	由 Outer 一致性层、Home UBCC 控制器、分层目录和 EP 边界组成的跨节点一致性体系结构	ubsim	
UBCC 控制器	维护全局目录、串行化同址事务并执行权限仲裁的控制器组件	ub	
Home UBCC 控制器	由地址映射选定、负责该地址全局目录和事务提交的 UBCC 控制器实例	Inner 域	节点内 gem5 CHI 一致性域
CHI	AMBA coherent transaction protocol；当前实现用于节点内 gem5 CHI 域，Outer CHI 仅作候选分析	Outer 域	Home UBCC 控制器协同实现的跨节点一致性层
HN-F	节点内 Home Node，负责本地一致性与内存访问	Home	负责指定地址全局目录和仲裁的节点
EP	节点内 CHI 域与 Outer 一致性层之间的端点扩展层	owner	持有写权限或最新脏数据的节点
EP-RNF	代表 Outer 域参与节点内 snoop 的端点	sharer	持有共享副本的节点
EP-SNF	将节点内服务请求接入 Outer 数据路径的端点	epoch	区分同址新旧事务的单调序号
UBAdapter	EP 与 UBIO 之间的消息适配组件	reqId	标识具体请求的事务编号
UBIO	承载 Home UBCC 控制器和分层目录的运行模块	Recall	从当前 owner 回收数据或权限
ResidentDir	SRAM 驻留目录	Invalidate	使共享副本失效
H64 Backstore	位于 metadata DRAM、保存冷目录元数据的 64 B bucket 哈希表	Clear	requester 本地完成后的提交确认
HA-VI	VI 协议可执行参考模型		